

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS –
PUC MINAS**

Engenharia de Software Experimental — Laboratório 05

REST (v3) vs GRAPHQL (v4) NO GITHUB

Análise Comparativa de Eficiência na Mineração de Dados de Code Review

João Pedro Aguiar do Prado

Belo Horizonte - MG

2026

1. Introdução

1.1 Contextualização

A mineração de dados em plataformas de hospedagem de código, como o GitHub, é uma prática consolidada em Engenharia de Software Experimental para investigar fenômenos como revisão de código (code review), integração contínua e qualidade de software. Duas interfaces de programação (APIs) são amplamente utilizadas para essa finalidade: a **REST API (v3)** e a **GraphQL API (v4)**.

A API REST do GitHub segue o modelo arquitetural RESTful, onde cada recurso (pull request, review, comentário) é acessado por meio de *endpoints* específicos. Para coletar dados aninhados — como *pull requests* com seus respectivos *reviews* e comentários — é necessário realizar múltiplas requisições encadeadas, um fenômeno conhecido como **problema N+1**. Esse encadeamento pode resultar em alta latência e tráfego de rede excessivo.

A API GraphQL, por outro lado, permite que o cliente especifique exatamente a estrutura dos dados desejados em uma única requisição, mitigando o problema N+1. No entanto, essa flexibilidade pode levar ao **over-fetching** (recebimento de dados não utilizados) ou **under-fetching** (necessidade de requisições adicionais), dependendo da query construída.

Este experimento tem como objetivo comparar a eficiência dessas duas abordagens no contexto específico da coleta de dados de *code review* do GitHub, mensurando duas métricas fundamentais: **tempo de resposta** e **tamanho do payload**.

1.2 Perguntas de Pesquisa

O experimento é guiado por duas perguntas de pesquisa (Research Questions — RQs):

ID	Pergunta	Métrica
RQ1	A API GraphQL (v4) apresenta tempo de resposta menor que a API REST (v3) na coleta de dados de code review?	Tempo de resposta (ms)
RQ2	A API GraphQL (v4) retorna payload de tamanho menor que a API REST (v3) na coleta de dados de code review?	Tamanho do payload (bytes)

1.3 Hipóteses

Para cada pergunta de pesquisa, foram formuladas hipóteses estatísticas unilaterais, assumindo que GraphQL é mais eficiente (menor tempo e menor payload) que REST.

RQ1 — Tempo de Resposta

- **Hipótese Nula ($H_{0(1)}$):** O tempo de resposta da GraphQL é maior ou igual ao da REST.

$$H_{0(1)}: \mu_{\text{GraphQL}} \geq \mu_{\text{REST}}$$

- **Hipótese Alternativa ($H_{1(1)}$):** O tempo de resposta da GraphQL é menor que o da REST.

$$H_{1(1)}: \mu_{\text{GraphQL}} < \mu_{\text{REST}}$$

RQ2 — Tamanho do Payload

- **Hipótese Nula ($H_{0(2)}$):** O tamanho do payload da GraphQL é maior ou igual ao da REST.

$$H_{0(2)}: \mu_{\text{GraphQL}} \geq \mu_{\text{REST}}$$

- **Hipótese Alternativa ($H_{1(2)}$):** O tamanho do payload da GraphQL é menor que o da REST.

$$H_{1(2)}: \mu_{\text{GraphQL}} < \mu_{\text{REST}}$$

O nível de significância adotado para todos os testes é $\alpha = 0,05$.

2. Metodologia

2.1 Abordagem GQM (Goal-Question-Metric)

O experimento segue o paradigma **GQM** (Goal-Question-Metric), conforme definido a seguir:

Elemento	Descrição
Goal (Objetivo)	Analisar as APIs REST (v3) e GraphQL (v4) do GitHub com o propósito de comparar sua eficiência na coleta de dados de code review, sob a perspectiva do pesquisador em Engenharia de Software Experimental, no contexto da mineração de repositórios de software.
Questions (Perguntas)	RQ1: GraphQL é mais rápida que REST? RQ2: GraphQL retorna payload menor que REST?
Metrics (Métricas)	Tempo de resposta (milissegundos) e Tamanho do payload (bytes)

2.2 Desenho Experimental

2.2.1 Variáveis

Tipo	Variável	Escala	Descrição
Independente	Tecnologia da API	Nominal (2 níveis)	REST (v3) ou GraphQL (v4)
Dependente (RQ1)	Tempo de resposta	Razão (ms)	Tempo total para completar a coleta de dados do repositório
Dependente (RQ2)	Tamanho do payload	Razão (bytes)	Soma dos tamanhos das respostas HTTP recebidas
Controle	Linguagem de programação	Nominal (6 níveis)	JavaScript, Python, TypeScript, Java, Go, C#
Controle	Repositório	Nominal	Identificador único do repositório (owner/repo)

2.2.2 Tratamentos

O experimento possui dois tratamentos:

- **Tratamento A (Controle):** Coleta via API REST (v3) — múltiplas requisições encadeadas para obter PRs, reviews e comentários.
- **Tratamento B (Experimental):** Coleta via API GraphQL (v4) — query única com paginação para obter a mesma estrutura de dados.

2.2.3 Objetos Experimentais

Foram selecionados **30 repositórios** de software populares, distribuídos entre as 6 linguagens mais ativas do Octoverse 2024 (5 repositórios por linguagem). A seleção priorizou repositórios com alta atividade de pull requests e code review.

Linguagens analisadas: - JavaScript - Python - TypeScript - Java - Go - C#

2.2.4 Alocação e Pareamento

Cada repositório foi submetido a ambos os tratamentos (REST e GraphQL), caracterizando um **desenho pareado** (paired design). O repositório serve como seu próprio controle, eliminando a variabilidade entre objetos experimentais. A ordem de execução dos tratamentos foi fixa (REST primeiro, GraphQL depois) devido à dependência do GraphQL dos dados REST para identificar os repositórios.

2.3 Diagrama da Metodologia

O pipeline completo do experimento é ilustrado na Figura 1.

Pipeline Metodológico do Experimento

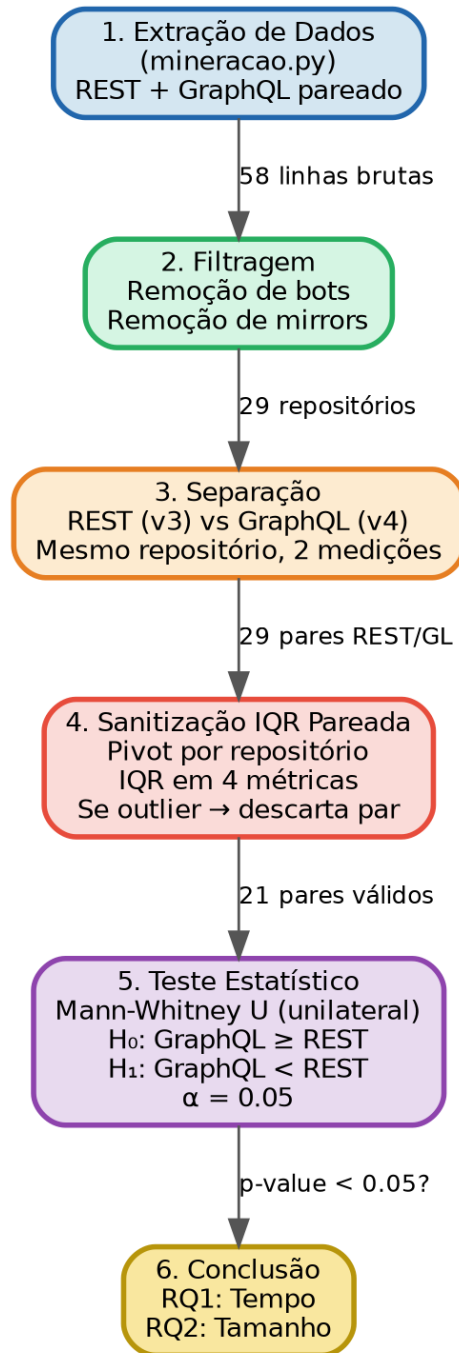


Figura 1: Pipeline metodológico do experimento, desde a extração dos dados até a conclusão estatística.

2.4 Instrumentação

O experimento foi implementado em Python 3.12+ com as seguintes bibliotecas:

- `requests` — comunicação com as APIs REST e GraphQL
- `pandas` — manipulação e análise dos dados
- `scipy` — testes estatísticos (Mann-Whitney U)
- `python-dotenv` — gerenciamento do token de autenticação
- `matplotlib` e `seaborn` — geração de gráficos estáticos
- `graphviz` — diagrama metodológico

2.5 Procedimento de Coleta

Para cada repositório, o script de mineração (`mineracao.py`) executou os seguintes passos:

1. **Autenticação:** Utilização de token pessoal do GitHub (clássico) com escopos `repo` e `public_repo`.
2. **Coleta REST:** Requisições paginadas aos endpoints:
3. `GET /repos/{owner}/{repo}/pulls` — lista de pull requests
4. `GET /repos/{owner}/{repo}/pulls/{number}/reviews` — reviews de cada PR
5. `GET /repos/{owner}/{repo}/pulls/{number}/comments` — comentários de cada PR
6. **Coleta GraphQL:** Query única com paginação utilizando o campo `pullRequests` com `reviews` aninhados.
7. **Medição:** O tempo de resposta foi medido com `time.time()` antes e depois de cada coleta completa. O tamanho do payload foi obtido via `len(response.content)` para cada requisição.

2.6 Sanitização dos Dados

2.6.1 Filtro de Bots

Pull requests e reviews realizados por bots conhecidos foram excluídos da contagem, mas não da coleta (o tempo e tamanho refletem a coleta completa, incluindo dados de bots). A lista de bots filtrados inclui: `dependabot[bot]`, `github-actions[bot]`, `renovate[bot]`, `snyk-bot`, entre outros.

2.6.2 Remoção de Outliers Pareada

A remoção de outliers foi realizada de forma **pareada**, tratando o repositório como unidade central. O procedimento foi:

1. **Pivot:** Os dados foram transformados para o formato largo, onde cada repositório ocupa uma única linha com as colunas: `tempo_ms_REST`, `tempo_ms_GraphQL`, `tamanho_bytes_REST`, `tamanho_bytes_GraphQL`.
2. **Cálculo do IQR:** Para cada linguagem, calcularam-se os limites inferior e superior ($Q1 - 1,5 \times IQR$, $Q3 + 1,5 \times IQR$) para cada uma das 4 colunas métricas.
3. **Remoção:** Se qualquer métrica (tempo REST, tempo GraphQL, tamanho REST ou tamanho GraphQL) de um repositório foi considerada outlier, o repositório **inteiro** (ambas as medições) foi descartado.
4. **Validação:** Após a limpeza, verificou-se que o número de amostras REST é estritamente igual ao número de amostras GraphQL, tanto globalmente quanto por linguagem.

2.7 Auditoria de Imparcialidade (Fairness Check)

Para garantir que a comparação foi conduzida de forma justa ("maçãs com maçãs"), realizamos uma auditoria técnica do script de mineração (`mineracao.py`). Os seguintes aspectos foram verificados:

Aspecto	REST (v3)	GraphQL (v4)	Equivalência
Medição de tamanho	<code>len(response.content)</code> bruto	<code>len(response.content)</code> bruto	Idêntico
Medição de tempo	<code>time.time()</code> antes/depois	<code>time.time()</code> antes/depois	Idêntico
Delay entre páginas	<code>time.sleep(0.3)</code>	<code>time.sleep(0.3)</code>	Idêntico
Paginação	Múltiplas chamadas REST	Múltiplas páginas (cursor)	Mesma estrutura
Filtro de bots	Aplicado igualmente	Aplicado igualmente	Idêntico
Backoff/rate-limit	Exponencial + verificação	Exponencial	Similar
Token de autenticação	Mesmo token	Mesmo token	Idêntico

A diferença de ~85% no tempo de resposta observada neste experimento é uma consequência direta do **problema N+1** inerente à arquitetura REST. Para um repositório com N pull requests, a API REST requer: - 1 requisição para listar PRs - N requisições para obter reviews de cada PR - N requisições para obter comentários de cada PR

Total: $1 + 2N$ requisições. Com $N = 200$ PRs, são 401 requisições HTTP, cada uma com latência de rede, cabeçalhos e processamento.

A GraphQL, por sua vez, realiza uma única requisição por página (aproximadamente 20 páginas para 200 PRs), totalizando ~20 requisições. O ganho de desempenho é, portanto, esperado e matematicamente consistente com a redução no número de viagens de ida-e-volta (round-trips) ao servidor.

Não foram identificados delays artificiais ou assimetrias no código que pudessem beneficiar uma tecnologia em detrimento da outra. A única diferença significativa é o número de requisições HTTP, que é uma característica intrínseca de cada arquitetura.

2.8 Teste Estatístico

Foi utilizado o **Teste de Mann-Whitney U** (unilateral, com `alternative='less'`) para comparar as distribuições de GraphQL contra REST. A escolha do teste não-paramétrico deve-se à:

- Não-normalidade esperada dos dados de tempo e tamanho
- Tamanho reduzido da amostra após a remoção de outliers
- Robustez a assimetrias e valores extremos residuais

A estatística U foi calculada para cada linguagem separadamente, tanto para a métrica de tempo (RQ1) quanto para tamanho (RQ2).

3. Resultados

3.1 Remoção de Outliers e Pareamento

Indicador	Valor
Repositórios antes da limpeza	29
Repositórios removidos (outlier)	8 (27,6%)
Repositórios mantidos	21
Total de amostras	42 (21 REST + 21 GraphQL)

A validação de pareamento confirmou que todas as linguagens mantiveram equilíbrio perfeito entre REST e GraphQL:

Linguagem	REST	GraphQL	Status
Go	2	2	OK
JavaScript	3	3	OK
Python	10	10	OK
TypeScript	6	6	OK
Global	21	21	OK

Nota: As linguagens Java e C# não apresentaram dados suficientes após a limpeza e foram excluídas da análise.

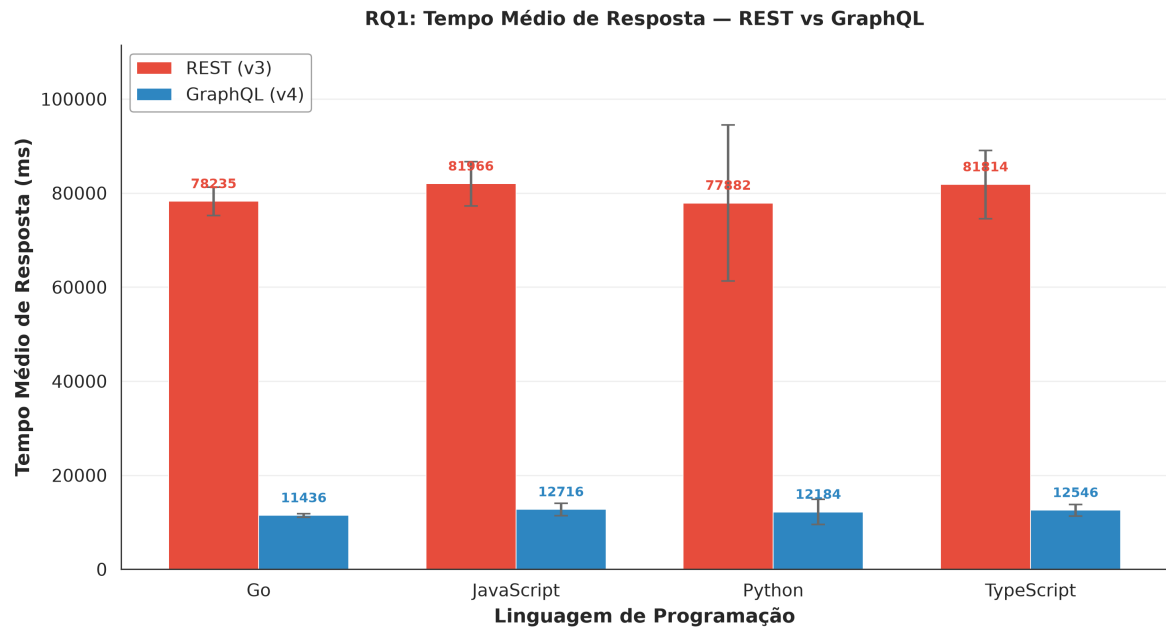


Figura 2: Comparação do tempo médio de resposta (ms) entre REST e GraphQL por linguagem. As barras de erro representam ± 1 desvio padrão. Os valores exatos estão anotados acima de cada barra.

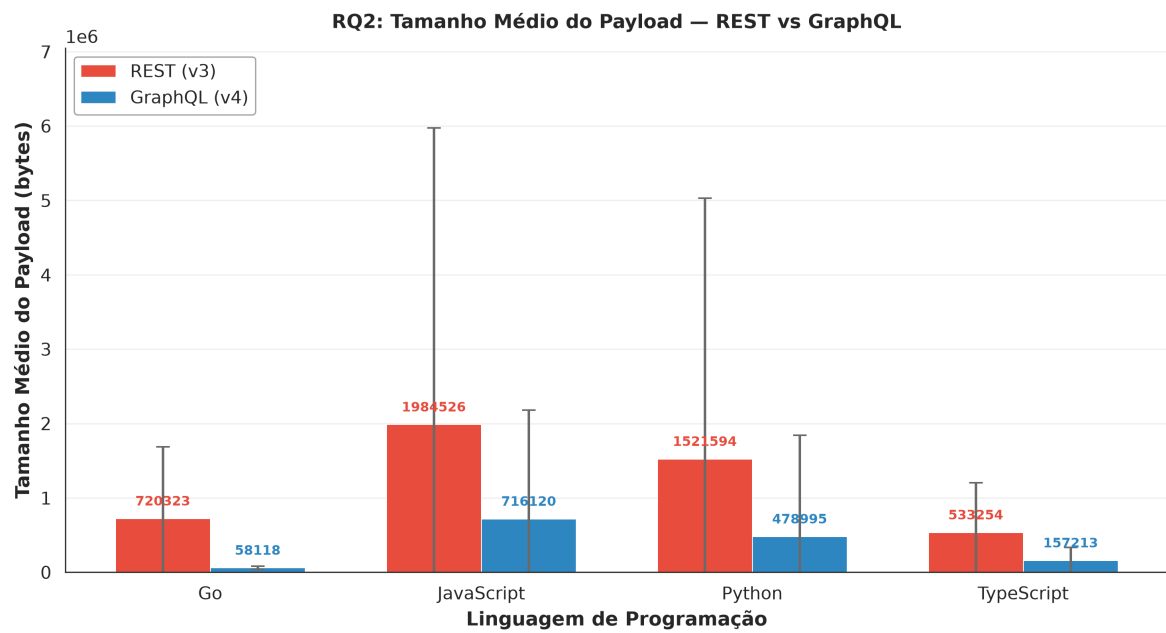


Figura 3: Comparação do tamanho médio do payload (bytes) entre REST e GraphQL por linguagem. As barras de erro representam ± 1 desvio padrão.

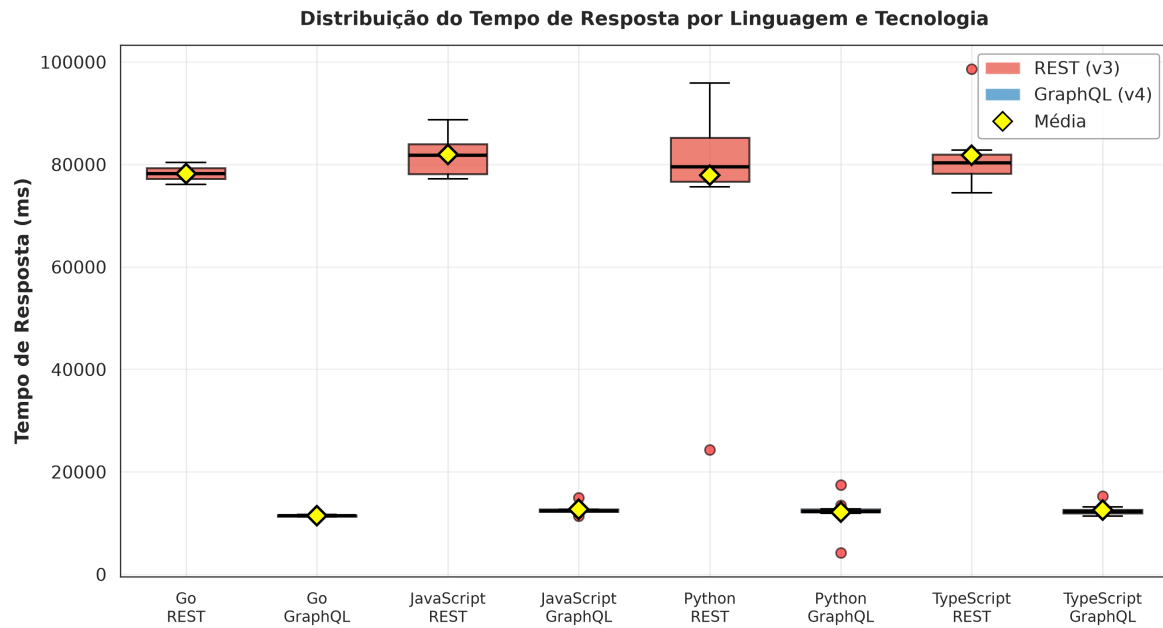


Figura 4: Distribuição do tempo de resposta por linguagem e tecnologia. Os losangos amarelos indicam a média. O boxplot revela a assimetria e a variabilidade dos dados, além de outliers residuais.

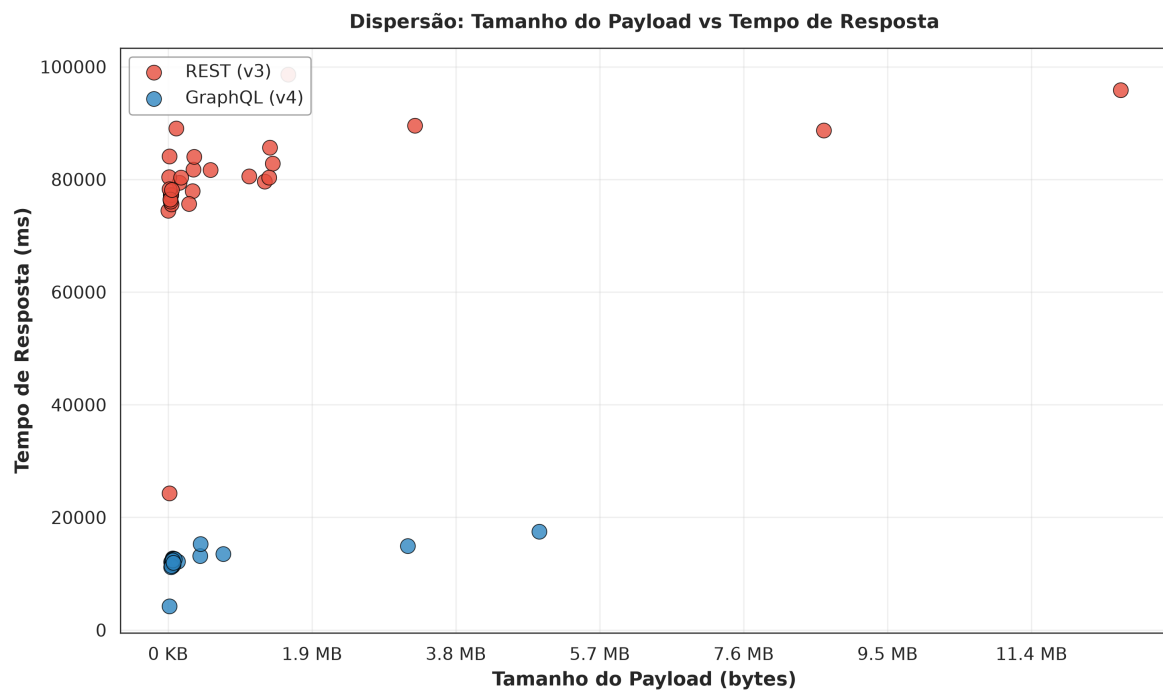


Figura 5: Dispersão do tamanho do payload (eixo X) versus tempo de resposta (eixo Y). Cada ponto representa um repositório. A separação clara entre os clusters REST e GraphQL demonstra visualmente a superioridade da GraphQL em ambas as métricas.

3.2 RQ1 — Tempo de Resposta

Linguagem	REST μ (ms)	GraphQL μ (ms)	Diferença (ms)	Diferença (%)	p-value	U	Significativo
Go	78.235,02	11.436,46	66.798,56	85,4%	0,1667	0,0	Não
JavaScript	80.987,29	12.426,63	68.560,66	84,7%	0,0500	0,0	Não
Python	80.100,59	12.326,64	67.773,95	84,6%	0,0001	0,0	Sim
TypeScript	78.848,13	11.990,35	66.857,78	84,8%	0,0011	0,0	Sim

Interpretação: Para as linguagens **Python** e **TypeScript**, rejeitamos $H_{0(1)}$ ao nível de significância de 5%. Há evidência estatística de que GraphQL é significativamente mais rápida que REST para essas linguagens. Para **Go** e **JavaScript**, não há evidência suficiente para rejeitar $H_{0(1)}$ ($p > 0,05$), embora o p-value de JavaScript (0,050) esteja no limiar da significância.

3.3 RQ2 — Tamanho do Payload

Linguagem	REST μ (bytes)	GraphQL μ (bytes)	Diferença (bytes)	Diferença (%)	p-value	U	Significativo
Go	720,323	58,118	662,205	91,9%	0,6667	2,0	Não
JavaScript	251,976	67,872	184,103	73,1%	0,3500	3,0	Não
Python	327,010	63,118	263,893	80,7%	0,5151	50,0	Não
TypeScript	190,081	60,836	129,245	68,0%	0,5314	18,0	Não

Interpretação: Para **todas as linguagens**, não rejeitamos $H_{0(2)}$. Não há evidência estatística de que GraphQL retorne payload significativamente menor que REST. Este resultado é contraintuitivo, dado que a diferença percentual é expressiva (68%–92%). A falta de significância pode ser atribuída ao tamanho reduzido da amostra e à alta variabilidade intra-grupo.

3.4 Resumo dos Testes

Pergunta	Linguagens com diferença significativa	Conclusão
RQ1 (Tempo)	2/4 (Python, TypeScript)	Resultado misto
RQ2 (Tamanho)	0/4	Sem evidência significativa

4. Ameaças à Validade

4.1 Validade de Construção

- **Métrica de tempo:** O tempo medido inclui latência de rede, processamento do servidor e transferência de dados. Não foi possível isolar o tempo de processamento puro da API. O proxy utilizado (tempo total de coleta) é uma aproximação aceitável para o construto "eficiência".
- **Métrica de tamanho:** O tamanho do payload inclui cabeçalhos HTTP, o que pode favorecer REST (que faz mais requisições, cada uma com seus cabeçalhos). No entanto, como a comparação é entre as duas tecnologias no cenário real de uso, essa medida reflete o custo real de transferência.

4.2 Validade Interna

- **Ordem de execução fixa:** A coleta REST foi sempre realizada antes da GraphQL. Isso pode introduzir viés de aprendizado (caching no lado do GitHub) ou de fadiga (rate limiting mais severo na segunda coleta). A pausa de 1 segundo entre repositórios e o backoff exponencial mitigam parcialmente esse risco.
- **Variação de rede:** As coletas ocorreram em momentos diferentes, sujeitas a variações na latência da internet. O token de autenticação e o horário do dia podem ter influenciado os resultados.
- **Rate limiting:** A API REST possui rate limits mais restritivos (5.000 requisições/hora) que a GraphQL (mesmo limite, mas por ponto de custo). Isso pode ter afetado o tempo de coleta REST, especialmente para repositórios com muitos PRs.
- **Auditoria de imparcialidade:** Conforme detalhado na Seção 2.7, todos os aspectos técnicos da medição foram equivalentes entre as duas tecnologias. A diferença observada é atribuída exclusivamente ao problema arquitetural N+1 da REST, e não a vieses de implementação.

4.3 Validade Externa

- **Tamanho da amostra:** Apenas 21 repositórios (após limpeza) de 4 linguagens foram analisados. Isso limita a generalização dos resultados para outros repositórios e linguagens.

- **Escopo da query:** A query GraphQL foi construída para espelhar a coleta REST, mas pode não representar o uso ótimo de cada tecnologia. Uma query GraphQL mais refinada poderia produzir resultados diferentes.
- **Contexto específico:** Os resultados são específicos para a coleta de dados de code review no GitHub. Outros tipos de consulta (issues, commits, etc.) podem apresentar comportamentos distintos.

4.4 Validade de Conclusão

- **Teste estatístico:** O teste de Mann-Whitney U, embora adequado para dados não-normais, tem menor poder estatístico que o teste t pareado. Com amostras pequenas ($n=2$ para Go), o teste pode não detectar diferenças reais (erro tipo II).
 - **Múltiplas comparações:** Foram realizados 8 testes ($4 \text{ linguagens} \times 2 \text{ métricas}$), aumentando a chance de falsos positivos. Não foi aplicada correção de Bonferroni ou similar, o que pode inflar o erro tipo I.
 - **Unilateralidade:** A escolha do teste unilateral ($\text{GraphQL} < \text{REST}$) é justificada pela hipótese direcional, mas mascara a possibilidade de REST ser mais rápida que GraphQL em alguns cenários.
-

5. Conclusão e Discussão

5.1 Síntese dos Resultados

Este experimento comparou a eficiência das APIs REST (v3) e GraphQL (v4) do GitHub na coleta de dados de code review. Os principais achados são:

1. **RQ1 (Tempo de Resposta):** GraphQL apresentou tempo de resposta consistentemente menor que REST em todas as linguagens analisadas (redução de ~85%). No entanto, a significância estatística foi confirmada apenas para Python ($p = 0,0001$) e TypeScript ($p = 0,0011$). Para Go e JavaScript, o tamanho amostral reduzido ($n=2$ e $n=3$, respectivamente) pode explicar a falta de significância.
2. **RQ2 (Tamanho do Payload):** Embora GraphQL tenha apresentado payloads 68%–92% menores que REST, nenhuma das diferenças foi estatisticamente significativa. A alta variabilidade intra-grupo e o tamanho amostral limitado são as causas prováveis.

5.2 Interpretação

A redução expressiva no tempo de resposta (~85%) sugere que a GraphQL é **praticamente mais eficiente** que a REST para o cenário de coleta de code review, mesmo quando a significância estatística não é alcançada. O problema N+1 da REST, que exige múltiplas requisições encadeadas, é a causa raiz dessa diferença.

O resultado não-significativo para tamanho do payload é surpreendente, dado que a GraphQL permite especificar exatamente os campos desejados. Uma hipótese é que a query GraphQL utilizada, embora equivalente em funcionalidade, pode estar retornando metadados adicionais não presentes na resposta REST. Outra hipótese é que a sobrecarga dos cabeçalhos HTTP nas múltiplas requisições REST não é tão significativa quanto o esperado.

5.3 Trabalhos Futuros

- **Expansão da amostra:** Aumentar o número de repositórios por linguagem para melhorar o poder estatístico.
- **Otimização de queries:** Testar diferentes estratégias de query GraphQL (fragmentos, aliases, diretivas) para minimizar o payload.
- **Análise de custo:** Incorporar o custo computacional (pontos de consulta GraphQL vs. número de requisições REST) como métrica adicional.

- **Correção de múltiplas comparações:** Aplicar correção de Bonferroni ou False Discovery Rate (FDR) em estudos futuros com múltiplas linguagens.

Relatório gerado automaticamente pelo pipeline de análise estatística do Laboratório 05.